

Assignment 1 – Entity-Component Framework [Part 1]

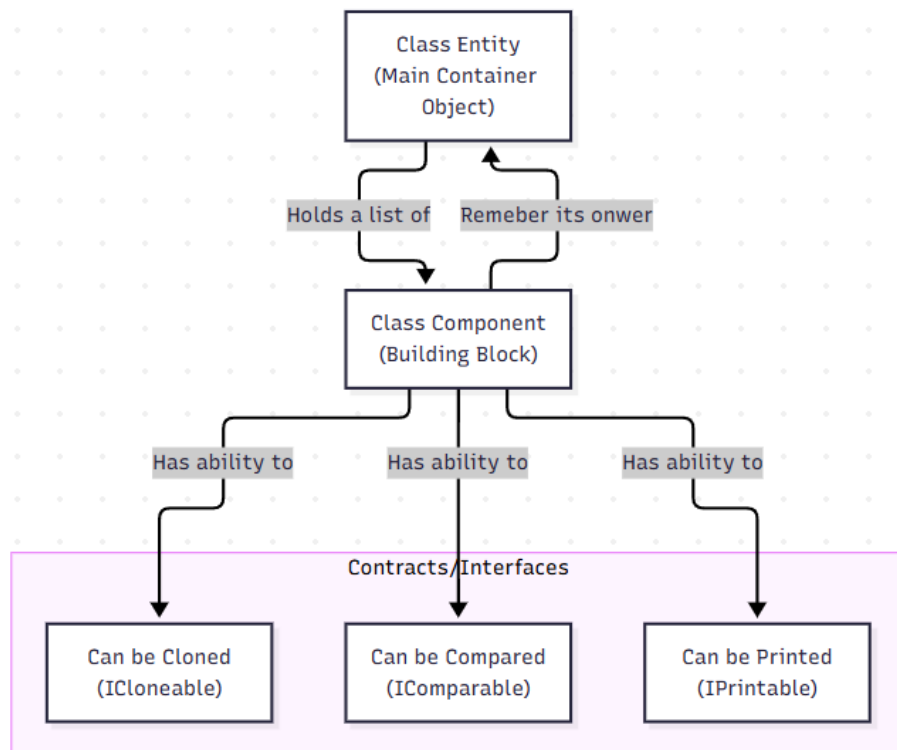
Goal

To put into practice the object-oriented programming principles and techniques covered in class, including inheritance, polymorphism, interfaces, downcasting, and upcasting. The goal of this assignment is to implement an Entity-Component framework for game development using an Entity-Component architecture. This framework is a fundamental building block of modern game architectures, so it is important that all students gain some experience with its internal design and implementation.

Students often find it challenging to express software design in terms of entities and components. This exercise provides an opportunity to work within a familiar domain while applying these concepts in a guided and controlled environment.

Description

The Entity-Component framework is one of the most fundamental building blocks of modern game engine architecture. The Entity-Component framework is a game architecture pattern in which an Entity is an empty container or object (e.g., a player or enemy), while Components provide modular chunks of data and behavior (e.g., Health, Transform, or Render). Entities are used to compose game objects by attaching Components, enabling a flexible, reusable, and extensible design without relying on deep inheritance hierarchies.



Component Class Requirements

Attributes	Description
Id	Integer storing the component identifier, can be inherited but not accessible outside
Entity	Pointer to owning Entity, can be inherited but not accessible outside
Methods	Description
get_id	Returns the component's id
clone	Creates and returns a deep copy of the component implementing interface ICloneable
compare_to	Compares this component to another implementing interface IComparable
print	Outputs a human-readable representation of the component implementing IPrintable interface (i.e. "This is a Component(with id=1)")
get_owner	Returns a pointer to the owning entity or null if orphan
set_owner	Assigns the owning Entity to this component

Entity class Requirements

Attributes	Description
Components	Internal container/array storing pointers of attached components
Methods	Description
component_count()	Returns the number of components attached or stored in the entity
attach (by pointer)	Attaches a component by pointer; must set the component's owner to this Entity
attach (by referen.)	Overloaded function where component can be attached by reference
operator[]	Overload operator that provides indexed access to components
Destructor	clean up all dynamically allocated components to prevent memory leaks

The first part of the assignment focuses on developing a functional framework. To accomplish this, students should follow the provided requirements and use both the test driver program (**test_driver.cpp**) and the test suite (**test_suite.hh**). Some tests only require the code to compile, while others require the implemented methods to perform the intended logic correctly.

For convenience, individual tests can be activated or deactivated by commenting or uncommenting the corresponding macros in the **test_helper.hh** file. You must start with the first tests and continue with the next one. If you get stuck in a particular test, just comment the macro and continue with another one.

The primary objective of this first part is to ensure that all tests compile and pass successfully. Code quality, documentation, and overall organization will also be evaluated. Students should not modify any files related to the tests; only the files specified for the assignment can be changed. Upon completion, students should have implemented a usable framework capable of representing an Entity-Component architecture.

What to do:

The code explicitly requires the following files, which the student should create and submit:

- component.hh and component.cc
- entity.hh and entity.cc

Submission Guidelines

Submit only componet.hh, component.cc, entity.hh and entity.cc as a zip folder on Moodle with the following naming conventions:

cs225fa26_{username}_1.1.zip